

PROYECTO FINAL — ENTREGA FINAL
Propuesta de Sistema de Base de Datos
Bases de Datos Avanzadas

1. INTEGRANTES DEL EQUIPO

#	Nombre completo	Correo institucional
1	Luisa Fernanda Guerrero Ordoñez	lfguerrero@udistrital.edu.co
2	Nelson David Posso Suárez	ndpossos@udistrital.edu.co
3	Mauricio Sánchez Aguilar	mausancheza@udistrital.edu.co
4	Oscar Santiago Duran Benitez	osduranb@udistrital.edu.co

2. TÍTULO DEL PROYECTO

Investigación UD

Link repositorio de Github:

<https://github.com/MauricioS98/DBA-Project.git>

3. DESCRIPCIÓN DE LA IDEA

¿Qué problema resuelve o qué proceso gestiona este sistema?

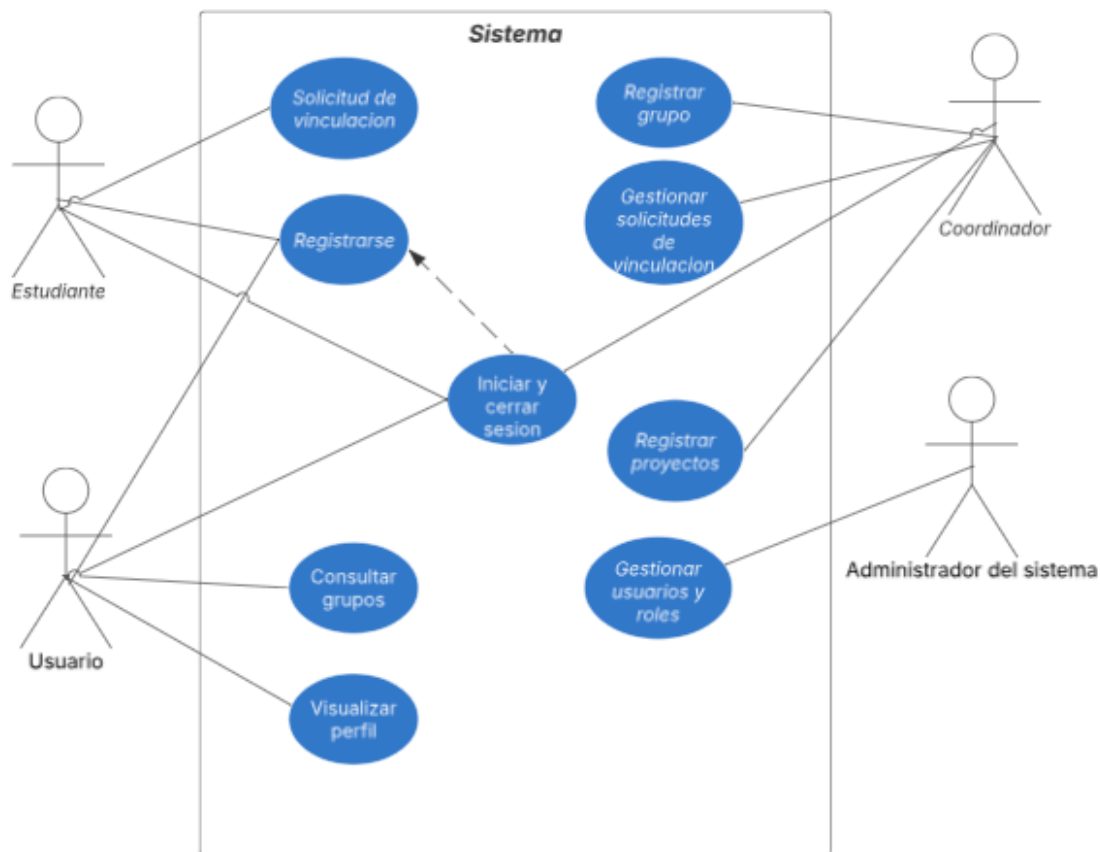
Investigación UD es un sistema de información orientado a la gestión y divulgación de los grupos de investigación de la Universidad Distrital. El sistema permite centralizar información sobre grupos, áreas, proyectos, productos académicos e integrantes, facilitando que estudiantes, docentes y usuarios externos consulten la actividad investigativa disponible.

Además, el sistema permite gestionar solicitudes de vinculación de estudiantes a grupos de investigación. Los estudiantes pueden enviar solicitudes, consultar su estado y conocer los proyectos disponibles, mientras que los coordinadores pueden revisar, aceptar o rechazar dichas solicitudes. De esta manera, el sistema apoya tanto la visibilidad de los grupos como la administración interna de sus procesos de vinculación.

4. USUARIOS DEL SISTEMA

Tipo de usuario	¿Qué acciones o consultas realizará?
Estudiante	Usuario principal que busca vincularse a grupos de investigación para participar en proyectos académicos.
Coordinador de grupo	Usuario principal que busca vincularse a grupos de investigación para participar en proyectos académicos.
Usuario general	Cualquier persona que accede al sistema para consultar información pública sobre los grupos
Administrador del sistema	Encargado de gestionar usuarios, roles y el funcionamiento general de la plataforma.

CASOS DE USO



Código	Caso de uso	Actor principal	Descripción breve
CU-01	Registrarse	Estudiante / Usuario	Permite que una persona cree una cuenta en el sistema ingresando sus datos personales, correo institucional y contraseña.

CU-02	Iniciar y cerrar sesión	Estudiante / Usuario / Coordinador / Administrador del sistema	Permite que los usuarios autenticados ingresen al sistema con sus credenciales y cierren sesión de forma segura.
CU-03	Solicitud de vinculación	Estudiante	Permite que el estudiante envíe una solicitud para vincularse a un grupo de investigación registrado en el sistema.
CU-04	Consultar grupos	Usuario	Permite visualizar el listado de grupos de investigación disponibles, junto con información general como nombre, área y descripción.
CU-05	Visualizar perfil	Usuario	Permite al usuario consultar la información asociada a su perfil dentro del sistema.
CU-06	Registrar grupo	Coordinador	Permite al coordinador registrar un nuevo grupo de investigación con su información básica, área, descripción y datos de contacto.
CU-07	Gestionar solicitudes de vinculación	Coordinador	Permite al coordinador revisar las solicitudes enviadas por los estudiantes y aceptarlas o rechazarlas según corresponda.
CU-08	Registrar proyectos	Coordinador	Permite al coordinador registrar proyectos de investigación asociados a un grupo, incluyendo título, descripción y estado.
CU-09	Gestionar usuarios y roles	Administrador del sistema	Permite al administrador crear, editar, eliminar o modificar los roles de los usuarios registrados en la plataforma.

5. REQUERIMIENTOS

Requerimientos Funcionales — ¿Qué debe hacer el sistema?		
#	Descripción del requerimiento	Prioridad (Alta / Media / Baja)
1	El sistema debe permitir que los usuarios se registren, inicien y cierren sesión mediante correo institucional y contraseña validando los campos antes de almacenar la información.	Alta
2	El sistema debe permitir que los coordinadores de grupos registren la información de sus grupos de investigación, incluyendo nombre, área, descripción, integrantes y proyectos.	Alta
3	El sistema debe mostrar un listado de todos los grupos de investigación registrados.	Media

4	El sistema debe permitir al estudiante enviar una solicitud de vinculación a un grupo de investigación.	Alta
5	El sistema debe permitir al coordinador visualizar las solicitudes pendientes a su grupo de investigación.	Alta
6	El sistema debe permitir al coordinador aceptar o rechazar solicitudes de ingreso a los grupos de investigación.	Alta
7	El sistema debe permitir al coordinador registrar proyectos de investigación con título, descripción, estado y fechas de inscripción.	Media
8	El sistema debe permitir visualizar toda la información de un grupo de investigación seleccionado.	Media
9	El administrador debe poder editar y eliminar usuarios.	Baja
10	El estudiante debe poder consultar el estado actual de su solicitud de vinculación a un grupo de investigación.	Media

Requerimientos No Funcionales — ¿Cómo debe comportarse?

#	Descripción del requerimiento	Prioridad (Alta / Media / Baja)
1	La aplicación debe ser visualmente coherente según el manual de marca.	Baja
2	El sistema debe implementar mecanismos de seguridad como validación de contraseñas, control de sesiones y acceso por roles.	Alta
3	El sistema debe mantener tiempos de respuesta menores a 10 segundos por solicitud	Media

Código	Requerimiento técnico	Estado	Evidencia en el proyecto
R1	Base de datos relacional	Cumple	Contamos con una base de datos relacional en PostgreSQL compuesta por más de cinco tablas relacionadas, entre ellas app_user, student, teacher, coordinator, investigation_area, investigation_team, investigation_project, product, product_type, application, product_student y product_teacher. Además, se implementan llaves primarias, llaves foráneas, restricciones NOT NULL y restricciones UNIQUE.
R2	Procedimiento almacenado	Cumple	Implementamos el procedimiento sp_aprobar_solicitud_vinculacion, el cual encapsula lógica de negocio relacionada con la aprobación de solicitudes de vinculación. Este procedimiento actualiza el estado de la solicitud y asocia al estudiante con el grupo de investigación correspondiente.

R3	Trigger	Cumple	Implementamos triggers de auditoría que se ejecutan automáticamente ante operaciones INSERT, UPDATE o DELETE. Por ejemplo, trg_audit_application permite registrar cambios realizados sobre las solicitudes, y también se incluyen funciones de auditoría para otras tablas del sistema.
R4	Transacción	Parcialmente cumple	Contamos con una operación crítica relacionada con la aprobación de solicitudes y vinculación del estudiante al grupo. Esta lógica se maneja dentro del procedimiento sp_aprobar_solicitud_vinculacion, incluyendo control de errores mediante EXCEPTION. Sin embargo, se recomienda evidenciar de forma explícita el uso de BEGIN, COMMIT y ROLLBACK desde la API o mediante un script de prueba.

6. ENTIDADES PRINCIPALES (PRELIMINAR)

Lista las entidades (objetos del mundo real) que creen que necesitarán en la base de datos. Esto puede cambiar en entregas posteriores.

#	Nombre de la entidad	Descripción breve / atributos principales
1	App_user	Datos de los usuarios(aquellos que se pueden loggear) / id, name, email, password, role
2	Application	Datos de las solicitudes para hacer parte de los grupos de investigación / id, user, investigation_team, state, app_date, app_msg, answ_date, answ_msg
3	Cordinator	Usuario especializado: Coordinador de un grupo de investigación / coordinador_id, teacher_id
4	Investigation_area	Áreas de investigación que tienen los proyectos curriculares / id, Project_area_id, name, descripcion
5	Investigation_project	Datos de los proyectos de investigación / id, team_id, title, resume, state
6	Investigation_team	Datos del grupo de investigación / id, área_id, cordinator_id, name, team_email, description

7	Product	Datos de los productos generados en el grupo de investigación / id, investigation_project_id, type_product_id, title, document, public_date
8	Product_type	Tabla con tipos de productos (artículos, libro, software, etc) / id, name, description
9	Project_area	Datos de los proyectos curriculares / project_area_id, name, Project_email
10	Student	Usuario especializado: Estudiante / user_id, student_id, team_id, Project_id, student_email
11	Teacher	Usuario especializado: Docente / user_id, teacher_id, team_id, Project_id, teacher_email
12	Product_student	Tabla de rompimiento entre producto y estudiante / product_id, student_id
13	Product_teacher	Tabla de rompimiento entre producto y profesor / product_id, teacher_id

Comportamiento del sistema

Propósito

El propósito de esta sección es describir cómo cambia el sistema en el tiempo a través de los principales procesos dinámicos del software. Se presentan los diagramas de secuencia correspondientes a las acciones complejas identificadas en la sección 6.3, con el fin de guiar la implementación de los métodos más relevantes por cada componente.

Estos diagramas muestran las interacciones entre los objetos del sistema, incluyendo componentes del backend (servicios, controladores, repositorios), el frontend y el sistema gestor de base de datos.

Procesos descritos

Los comportamientos modelados corresponden a las acciones complejas del sistema:

1. Registro e inicio de sesión del usuario
2. Solicitud de vinculación por parte del estudiante
3. Gestión de solicitudes por parte del coordinador

Diagrama de secuencia — Registro de Usuario (CU-01)

Descripción:

Este proceso permite al usuario crear una cuenta proporcionando sus datos personales. El sistema valida la información e inserta el nuevo registro en la base de datos.

Participantes:

- Usuario
- Frontend Angular
- AuthController
- AuthService
- UsuarioRepository
- Base de datos PostgreSQL

Secuencia:

1. El usuario accede al formulario y envía datos de registro.
2. El frontend realiza una petición HTTP POST al AuthController.
3. AuthService valida datos y verifica que el correo no exista.
4. UsuarioRepository almacena el nuevo usuario en la base de datos.
5. Se retorna confirmación al usuario.

Diagrama (texto UML):

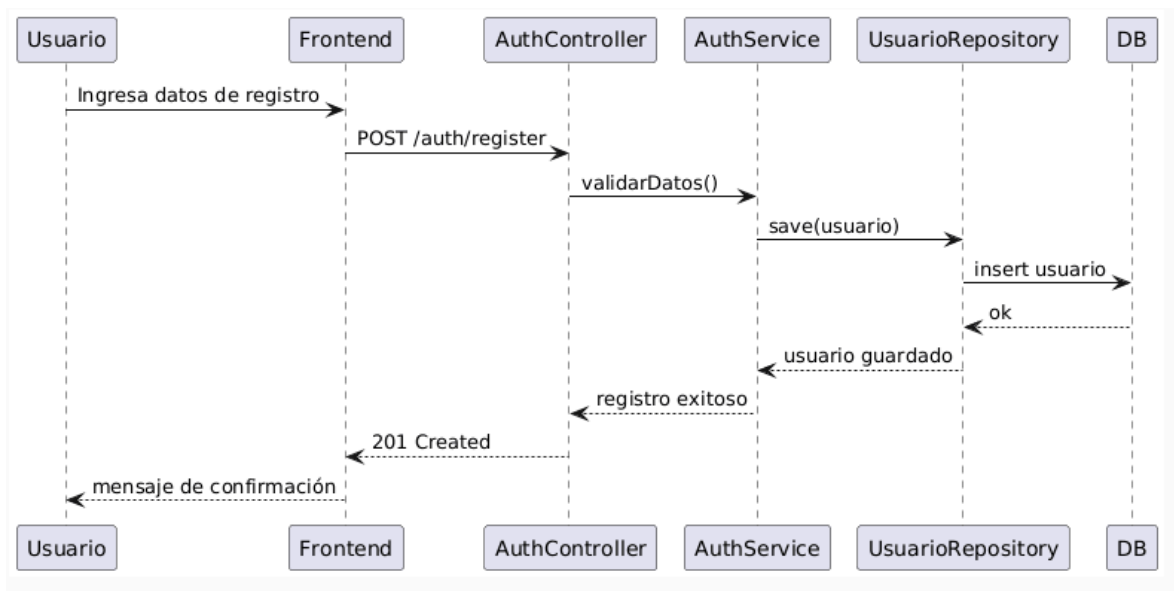


Diagrama de secuencia Registro de Usuario

Diagrama de secuencia — Inicio de Sesión (CU-09)

Descripción:

El usuario ingresa credenciales, el sistema valida y genera un token de autenticación.

Participantes:

- Usuario
- Frontend Angular
- AuthController
- AuthService
- UsuarioRepository
- Módulo JWT

Diagrama:

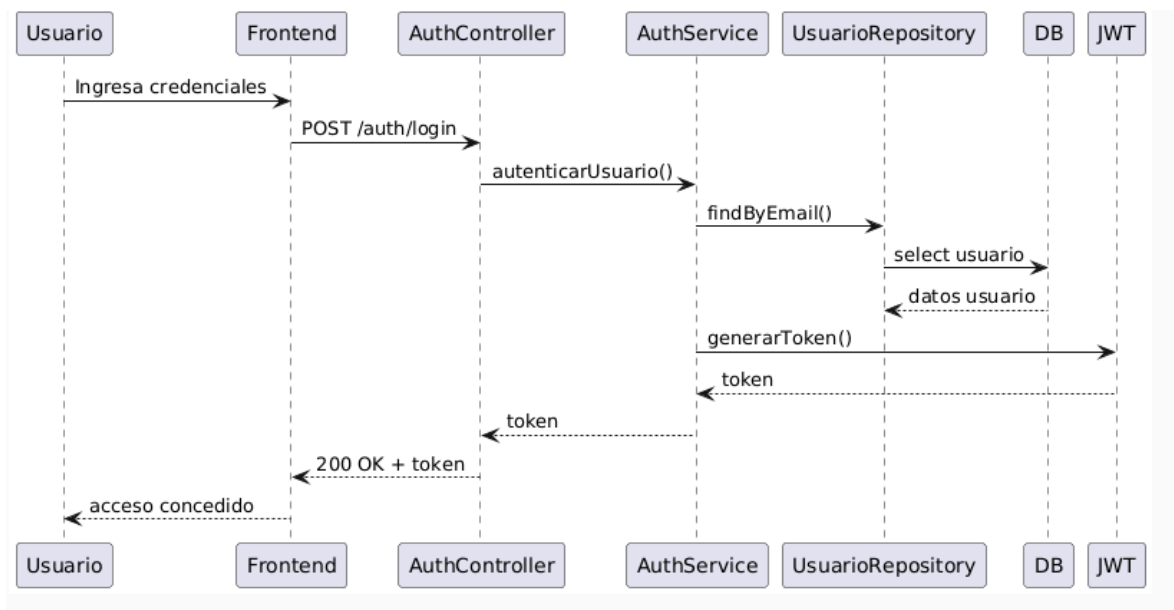


Diagrama de Secuencia Inicio de Sesión

Diagrama de secuencia — Solicitud de Vinculación (CU-04)

Descripción:

El estudiante envía una solicitud para ingresar a un grupo de investigación.

Participantes:

- Estudiante
- Frontend Angular
- SolicitudesController
- SolicitudesService
- SolicitudRepository
- Base de datos PostgreSQL

Diagrama:

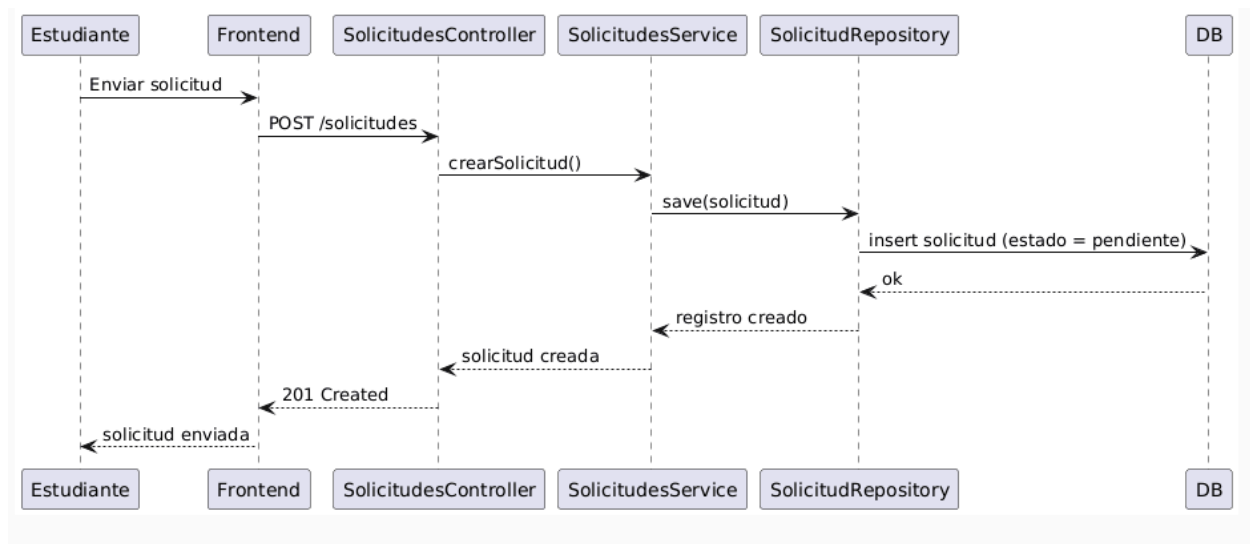


Diagrama de secuencia Solicitud de Vinculación

Diagrama de secuencia — Gestión de Solicitudes (CU-05)

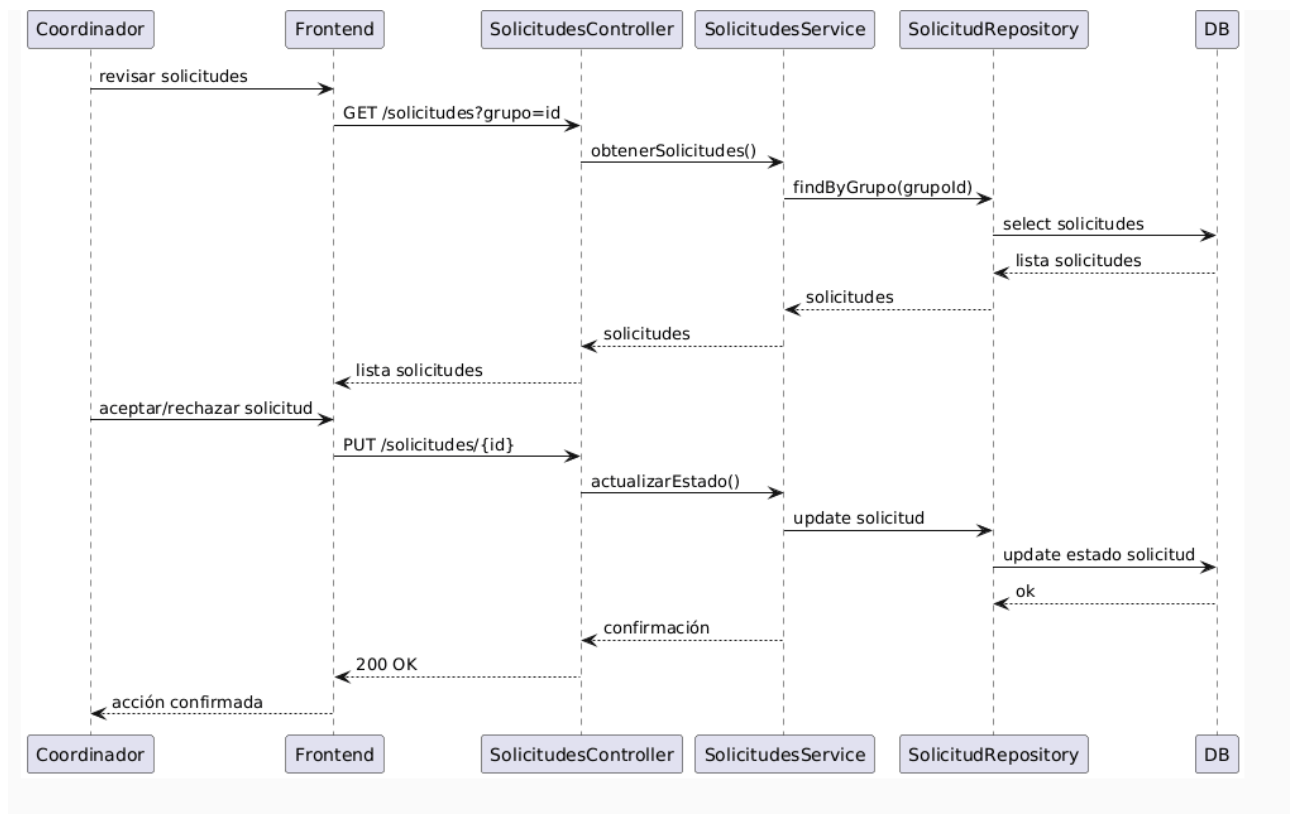
Descripción:

El coordinador revisa las solicitudes y decide aceptar o rechazar la postulación del estudiante.

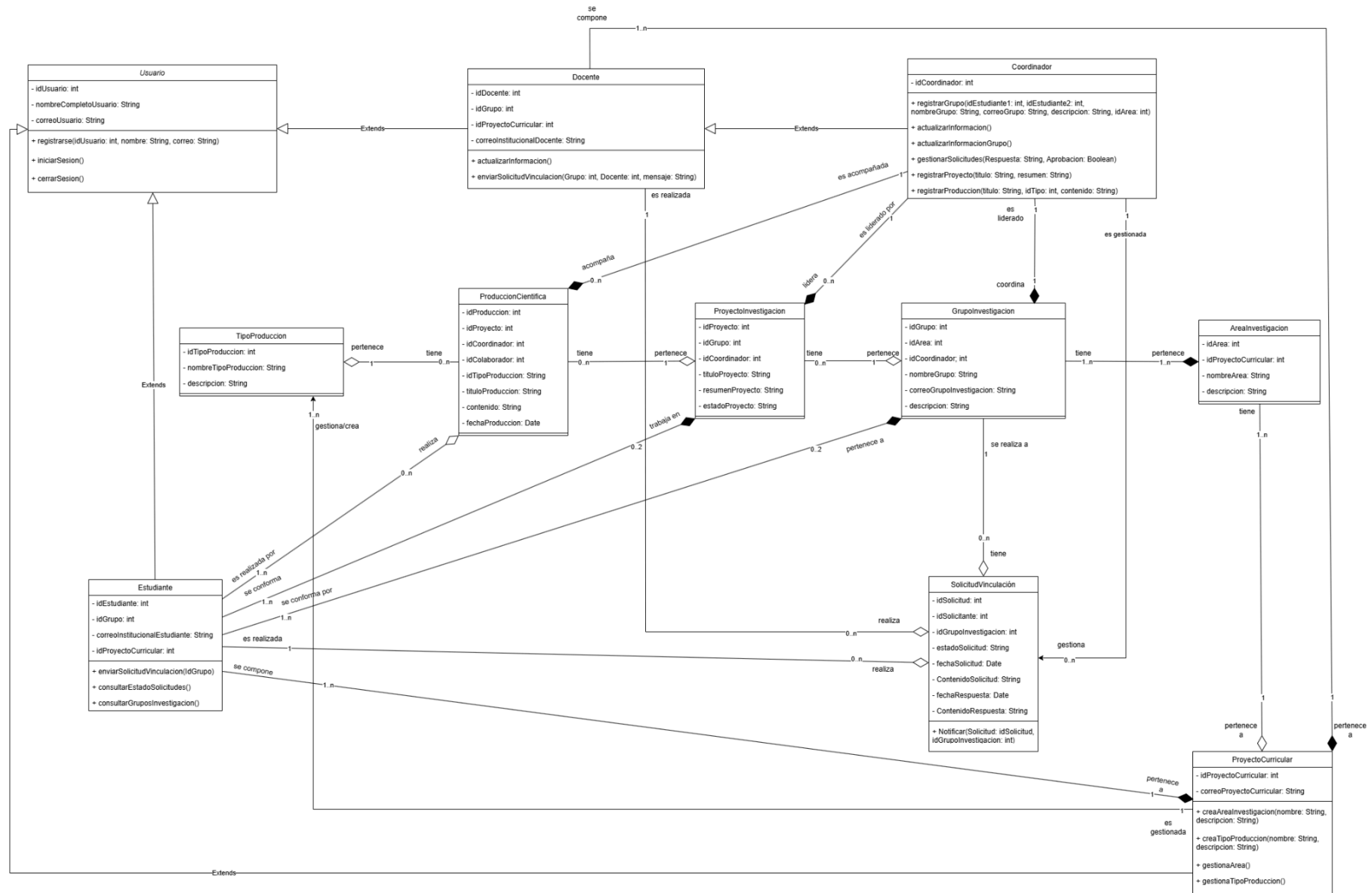
Participantes:

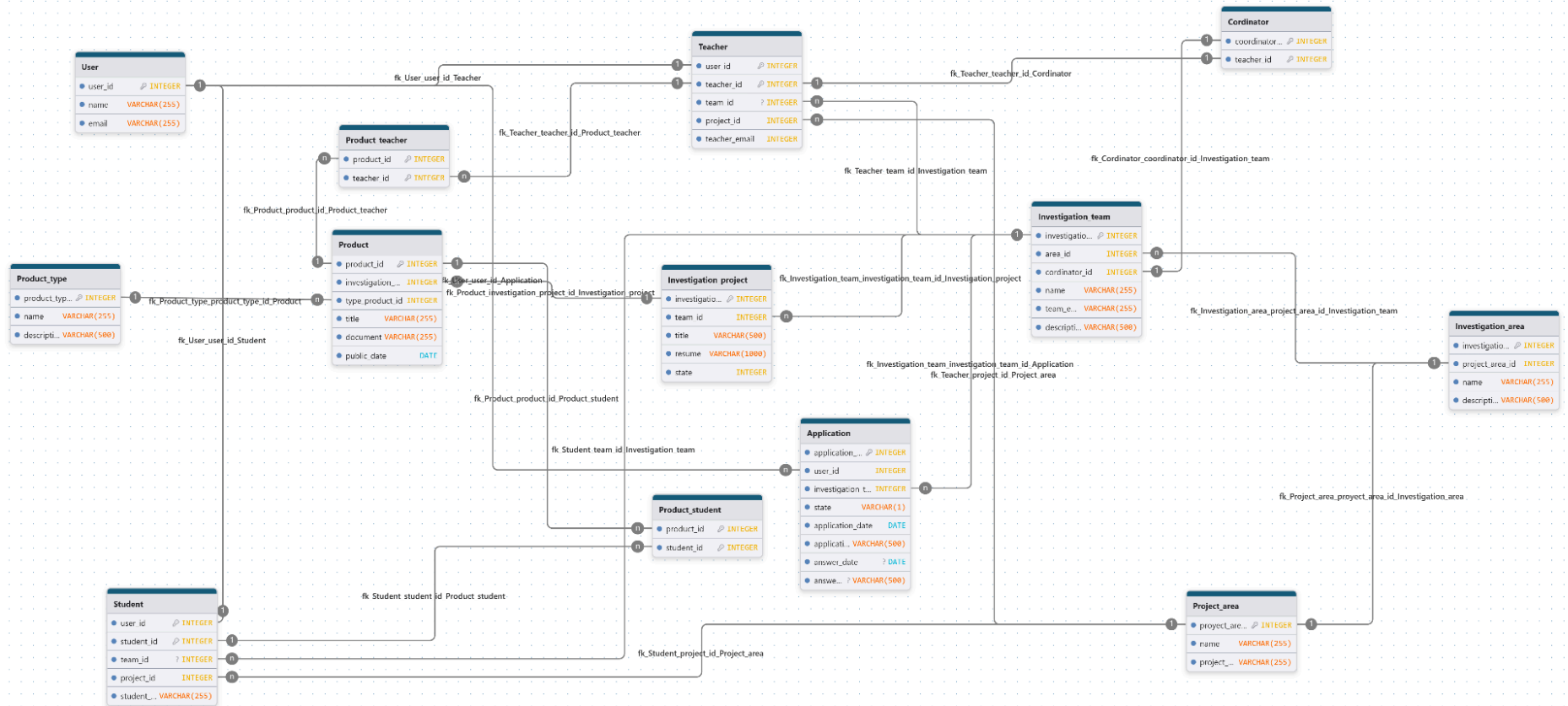
- Coordinador
- Frontend Angular
- SolicitudesController
- SolicitudesService
- SolicitudRepository
- UsuarioRepository (si se actualiza estado del estudiante)
- Base de datos PostgreSQL

Diagrama:

**Diagrama de Secuencia Gestión de Solicitudes**

Persistencia





CREACIÓN DE TABLAS

```
CREATE TABLE App_user (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    role VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Project_area (  
    project_area_id SERIAL PRIMARY KEY,  
    name VARCHAR(150) NOT NULL,  
    project_email VARCHAR(150)  
);
```

```
CREATE TABLE Investigation_area (  
    id SERIAL PRIMARY KEY,  
    project_area_id INTEGER NOT NULL,  
    name VARCHAR(150) NOT NULL,  
    descripcion TEXT  
);
```

```
CREATE TABLE Investigation_team (  
    id SERIAL PRIMARY KEY,  
    area_id INTEGER NOT NULL,  
    coordinator_id INTEGER,  
    name VARCHAR(150) NOT NULL,  
    team_email VARCHAR(150),  
    description TEXT
```

);

```
CREATE TABLE Investigation_project (  
    id SERIAL PRIMARY KEY,  
    team_id INTEGER NOT NULL,  
    title VARCHAR(200) NOT NULL,  
    resume TEXT,  
    state VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Product_type (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    description TEXT  
);
```

```
CREATE TABLE Product (  
    id SERIAL PRIMARY KEY,  
    investigation_project_id INTEGER NOT NULL,  
    type_product_id INTEGER NOT NULL,  
    title VARCHAR(200) NOT NULL,  
    document TEXT,  
    public_date DATE  
);
```

```
CREATE TABLE Student (  
    user_id INTEGER NOT NULL,  
    student_id SERIAL PRIMARY KEY,  
    team_id INTEGER,  
    project_area_id INTEGER,  
    student_email VARCHAR(150)
```

);

```
CREATE TABLE Teacher (  
    user_id INTEGER NOT NULL,  
    teacher_id SERIAL PRIMARY KEY,  
    team_id INTEGER,  
    project_area_id INTEGER,  
    teacher_email VARCHAR(150)  
);
```

```
CREATE TABLE Cordinator (  
    coordinador_id SERIAL PRIMARY KEY,  
    teacher_id INTEGER NOT NULL,  
    investigation_team_id INTEGER NOT NULL  
);
```

```
CREATE TABLE Application (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER NOT NULL,  
    investigation_team_id INTEGER NOT NULL,  
    state VARCHAR(50) NOT NULL,  
    app_date TIMESTAMP NOT NULL DEFAULT NOW(),  
    app_msg TEXT,  
    answ_date TIMESTAMP,  
    answ_msg TEXT  
);
```

```
CREATE TABLE Product_student (  
    product_id INTEGER NOT NULL,  
    student_id INTEGER NOT NULL,  
    PRIMARY KEY (product_id, student_id)
```


);

```
CREATE TABLE Product_teacher (  
    product_id INTEGER NOT NULL,  
    teacher_id INTEGER NOT NULL,  
    PRIMARY KEY (product_id, teacher_id)  
);
```

– Alter tables

-- Investigation_area

```
ALTER TABLE Investigation_area  
ADD CONSTRAINT fk_investigation_area_project_area  
FOREIGN KEY (project_area_id) REFERENCES Project_area(project_area_id);
```

-- Investigation_team

```
ALTER TABLE Investigation_team  
ADD CONSTRAINT fk_investigation_team_area_id FOREIGN KEY (area_id) REFERENCES  
Investigation_area(id),  
ADD CONSTRAINT fk_investigation_team_cordinator_id FOREIGN KEY (cordinator_id) REFERENCES  
Cordinator(coordinador_id);
```

-- Investigation_project

```
ALTER TABLE Investigation_project  
ADD CONSTRAINT fk_investigation_project_team_id  
FOREIGN KEY (team_id) REFERENCES Investigation_team(id);
```

-- Product

```
ALTER TABLE Product  
ADD CONSTRAINT fk_product_investigation_project FOREIGN KEY (investigation_project_id)  
REFERENCES Investigation_project(id),  
ADD CONSTRAINT fk_product_type_product FOREIGN KEY (type_product_id) REFERENCES  
Product_type(id);
```

-- Student

ALTER TABLE Student

ADD CONSTRAINT fk_student_user_id FOREIGN KEY (user_id) REFERENCES App_user(id),

ADD CONSTRAINT fk_student_team_id FOREIGN KEY (team_id) REFERENCES
Investigation_team(id),

ADD CONSTRAINT fk_student_project_area_id FOREIGN KEY (project_area_id) REFERENCES
Project_area(project_area_id);

-- Teacher

ALTER TABLE Teacher

ADD CONSTRAINT fk_teacher_user_id FOREIGN KEY (user_id) REFERENCES App_user(id),

ADD CONSTRAINT fk_teacher_team_id FOREIGN KEY (team_id) REFERENCES
Investigation_team(id),

ADD CONSTRAINT fk_teacher_project_area_id FOREIGN KEY (project_area_id) REFERENCES
Project_area(project_area_id);

-- Cordinator

ALTER TABLE Cordinator

ADD CONSTRAINT fk_cordinator_teacher_id FOREIGN KEY (teacher_id) REFERENCES
Teacher(teacher_id),

ADD CONSTRAINT fk_cordinator_team_id FOREIGN KEY (investigation_team_id) REFERENCES
Investigation_team(id);

-- Application

ALTER TABLE Application

ADD CONSTRAINT fk_application_user_id FOREIGN KEY (user_id) REFERENCES App_user(id),

ADD CONSTRAINT fk_application_investigation_team_id FOREIGN KEY (investigation_team_id)
REFERENCES Investigation_team(id);

-- Product_student

ALTER TABLE Product_student

ADD CONSTRAINT fk_product_student_product_id FOREIGN KEY (product_id) REFERENCES
Product(id),

ADD CONSTRAINT fk_product_student_student_id FOREIGN KEY (student_id) REFERENCES
Student(student_id);

```
-- Product_teacher
```

```
ALTER TABLE Product_teacher
```

```
ADD CONSTRAINT fk_product_teacher_product_id FOREIGN KEY (product_id) REFERENCES  
Product(id),
```

```
ADD CONSTRAINT fk_product_teacher_teacher_id FOREIGN KEY (teacher_id) REFERENCES  
Teacher(teacher_id);
```

```
-- TRIGGERS Y TABLA DE AUDITORÍA
```

```
-- Esta sección registra automáticamente los INSERT, UPDATE y DELETE
```

```
-- realizados sobre la tabla Application.
```

```
CREATE TABLE application_audit (
```

```
    audit_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    application_id INTEGER,
```

```
    user_id INTEGER,
```

```
    investigation_team_id INTEGER,
```

```
    old_state VARCHAR(32),
```

```
    new_state VARCHAR(32),
```

```
    old_answer_message VARCHAR(2000),
```

```
    new_answer_message VARCHAR(2000),
```

```
    operacion CHAR(1),
```

```
    fecha_cambio TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

```
CREATE OR REPLACE FUNCTION fn_audit_application()
RETURNS TRIGGER

LANGUAGE plpgsql

AS $$

BEGIN

    IF (TG_OP = 'INSERT') THEN

        INSERT INTO application_audit (application_id, user_id, investigation_team_id, new_state,
new_answer_message, operacion)

        VALUES (NEW.application_id, NEW.user_id, NEW.investigation_team_id, NEW.state,
NEW.answer_message, 'I');

        RETURN NEW;

    ELSIF (TG_OP = 'UPDATE') THEN

        INSERT INTO application_audit (application_id, user_id, investigation_team_id, old_state, new_state,
old_answer_message, new_answer_message, operacion)

        VALUES (OLD.application_id, OLD.user_id, OLD.investigation_team_id, OLD.state, NEW.state,
OLD.answer_message, NEW.answer_message, 'U');

        RETURN NEW;

    ELSIF (TG_OP = 'DELETE') THEN

        INSERT INTO application_audit (application_id, user_id, investigation_team_id, old_state,
old_answer_message, operacion)

        VALUES (OLD.application_id, OLD.user_id, OLD.investigation_team_id, OLD.state,
OLD.answer_message, 'D');

        RETURN OLD;

    END IF;

    RETURN NULL;

END;
```

```
$$;

CREATE TRIGGER trg_audit_application
AFTER INSERT OR UPDATE OR DELETE ON Application
FOR EACH ROW EXECUTE FUNCTION fn_audit_application();

-- PROCEDIMIENTO ALMACENADO

-- Este procedimiento aprueba una solicitud de vinculación y asigna
-- el estudiante al equipo de investigación correspondiente.

CREATE OR REPLACE PROCEDURE sp_aprobar_solicitud_vinculacion(
    p_application_id INT,
    p_answer_message VARCHAR(2000)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_user_id INT;
    v_team_id INT;
    v_current_state VARCHAR(32);
BEGIN
    -- 1. Obtener la información de la solicitud y validar su existencia

    SELECT user_id, investigation_team_id, state
    INTO v_user_id, v_team_id, v_current_state
    FROM Application
    WHERE application_id = p_application_id;
```

```
IF NOT FOUND THEN

    RAISE EXCEPTION 'Error: La solicitud con ID % no existe.', p_application_id;

END IF;

-- 2. Validar que la solicitud no haya sido procesada previamente

IF v_current_state <> 'Pendiente' THEN

    RAISE EXCEPTION 'Error: La solicitud ya se encuentra en estado % y no puede ser modificada.',
v_current_state;

END IF;

-- 3. Actualizar el estado de la postulación en la tabla Application

UPDATE Application

SET state = 'Aprobado',

    answer_date = CURRENT_DATE,

    answer_message = p_answer_message

WHERE application_id = p_application_id;

-- 4. Vincular al estudiante al equipo de investigación (Actualizar tabla Student)

-- Nota: Se busca en la tabla Student usando el user_id heredado de app_user

UPDATE Student

SET team_id = v_team_id

WHERE user_id = v_user_id;

-- Si el usuario que postuló no era un Estudiante (o no se encontró en la tabla)

IF NOT FOUND THEN
```

```
RAISE EXCEPTION 'Error de integridad: El usuario asociado a la solicitud no está registrado como
Estudiante.';

END IF;

-- Si fue exitoso PostgreSQL realiza el COMMIT implícito al finalizar el bloque.

RAISE NOTICE 'Transacción exitosa: Solicitud % aprobada y estudiante vinculado al equipo %.',
p_application_id, v_team_id;

EXCEPTION

-- En caso de cualquier error se captura, se fuerza un ROLLBACK automático y se informa

WHEN OTHERS THEN

    RAISE NOTICE 'ROLLBACK EJECUTADO: Se detectó un error en la operación. Detalles: %', SQLERRM;

    RAISE;

END;

$$;
```

CONCLUSIONES

El desarrollo del sistema Investigación UD permitió diseñar una solución orientada a la gestión y divulgación de grupos de investigación, integrando información sobre usuarios, estudiantes, docentes, coordinadores, áreas, proyectos, productos académicos y solicitudes de vinculación. Con esto, el sistema busca apoyar un proceso real dentro del contexto universitario, facilitando tanto la consulta pública de información como la administración interna de los grupos.

A nivel de base de datos, se logró construir un modelo relacional en PostgreSQL con varias tablas relacionadas mediante llaves primarias y foráneas, lo que permite organizar la información de forma estructurada y mantener la integridad de los datos. Además, el uso de restricciones como NOT NULL y UNIQUE contribuye a evitar registros incompletos o duplicados en entidades importantes como los usuarios.

También se incorporaron elementos propios de bases de datos avanzadas, como procedimientos almacenados y triggers. El procedimiento almacenado permite manejar la lógica de aprobación de solicitudes de vinculación, mientras que los triggers de auditoría ayudan a registrar cambios importantes realizados sobre las tablas del sistema. Esto permite mejorar el control, seguimiento y confiabilidad de las operaciones realizadas en la base de datos.

Finalmente, aunque el proyecto cumple con varios componentes importantes, también se identifican oportunidades de mejora para futuras versiones, como fortalecer el uso de vistas, subconsultas, índices y transacciones explícitas desde la API. Estas mejoras permitirían optimizar consultas, facilitar reportes, mejorar el rendimiento y garantizar mayor consistencia en operaciones críticas del sistema.